Slide 1: DSASSASSIN

Hi everyone. We're Siddharth Banakar and Dinesh Samala, and today we're presenting DSASSASSIN. The paper studies Intel's Data Streaming Accelerator, or DSA, which moves memory operations away from the CPU.

The question is fairly simple. If two virtual machines share that accelerator, can one learn something about the other's activity without reading its memory? The authors show two ways that this can happen. We'll explain the hardware first, walk through the experiments and code, and then show how those observations become application-level inferences.

Slide 2: Presentation plan

Here's the route we'll take. First, we'll look at why memory operations are offloaded, then follow a DSA request and its address translations. That gives us the pieces we need to understand what the virtual machines share.

Then we'll look at the experiments and the two attacks. After that, we'll follow a probe result into a trace and an inference. We'll finish with the application results, the evaluation limits, and the proposed mitigations. The first question is why these memory operations are being sent to an accelerator at all.

Slide 3: Why offload memory operations?

Before we look at the attack, why use DSA at all? A CPU can already copy and compare memory, but moving large buffers repeatedly uses instruction-execution time and ties up cores that could be doing application work.

The program selects these bulk jobs; ordinary CPU loads and stores still execute on the CPU. DSA takes those bulk operations as asynchronous jobs. While it handles a buffer, the CPU can work on something else, provided the program doesn't need that result yet. That can help with network buffers, storage, and virtual-machine checkpoints.

The supported jobs include copies and dualcast, which copies one source to two destinations; filling and comparing memory; CRC checksums and DIF integrity operations; and creating or applying delta records.

There is still submission and completion overhead, so small jobs aren't automatically faster when offloaded. DSA also doesn't create extra DRAM bandwidth. The benefit depends on the operation and workload. The security question is what happens when different tenants share the hardware doing that work.

Slide 4: Private addresses can still share hardware

Imagine two virtual machines using the same physical accelerator. Their memory is protected, so the attacker can't simply load an address and read the victim's data. Requests carry an address-space identity, and the translation system checks which memory they may access.

But address permissions don't automatically separate every internal hardware resource. Both VMs may still use the same engine or compete for admission to the same queue. The attacker can submit its own requests and observe whether they were accepted or how long they took to finish.

That's the distinction to keep in mind throughout this talk. The observation is about shared resource activity. Any later claim about a website or a keystroke has to be inferred from that observation. To see where those signals come from, let's follow a DSA request.

Slide 5: How one DSA request completes

A program starts by preparing a descriptor. Think of it as a small request record containing the operation, the relevant addresses, and where to report completion. The program submits that descriptor through a portal, and the work queue buffers it for an engine.

On the original diagram, follow the request into the processing unit. The stages are decode, read, compute, write, and notify. The engine may need address translations before issuing memory transactions. Once the work is complete, the completion record tells the submitting program that it can continue.

There are two different moments to observe here. A submission can be accepted before the operation has finished. So the attacker can ask, 'Did my request get into the queue?' It can also ask, 'How long did my request take to complete?' Those become the two channels. First, we need to understand the translations used along this path.

Slide 6: How IOMMU and VT-d protect device accesses

Now let's make the address-translation path concrete. An IOMMU translates device memory requests and checks whether those accesses are allowed. Intel VT-d is Intel's architecture for this I/O translation and isolation.

A request is interpreted in its device and address-space context. PASID identifies a process address space within that context; it isn't a globally meaningful identity by itself. The translation maps the requested address to a physical address and supplies the relevant access permissions. Under virtualization, that may involve multiple translation stages rather than one flat lookup.

DSA keeps reusable translations in its device-side cache, the DevTLB. On a hit, it reuses an authorized cached mapping. On a miss, it sends an ATS translation request to the host's Translation Agent, associated with the IOMMU. The host may answer from its IOTLB cache. Otherwise, it may need a page-table walk to resolve the request.

The physical page is simply a region of RAM backing the data, perhaps a source buffer, destination buffer, or completion record. Physical describes its address, not a new type of storage. The DevTLB holds mapping information, context, and permissions, not those bytes or an application name. Let's now look at how many translations each engine retains.

Slide 7: Five translation slots per engine

The authors reverse-engineered the structure shown here. On their tested hardware, each engine has five translation slots. They correspond to the source, second source, destination, second destination, and completion-record fields. Each slot retains one translation.

That small capacity makes replacement easy to observe. Suppose my completion record is on page A, and its translation is currently in the completion slot. A later completion write using another page can replace it. When I access my own page again, I may have to fetch its translation again, which can make my request slower.

Notice what I haven't learned. I still don't know the other page's address or contents. I have evidence that the shared state changed. Also, this is the organization measured on the tested platforms, not a claim about every possible accelerator design. The next question is how a descriptor selects one of these slots.

Slide 8: Descriptor fields select the slot

Here's the descriptor format from the paper. The important point is that the operation determines what each address field means. A copy uses source and destination addresses. A compare uses two source addresses. With completion writing enabled, both also use the completion-record address.

Look at offset 24. For a compare operation, those bytes represent the second source. For a copy, the same bytes represent the destination. The researchers found that these roles use separate translation slots, even though they share a position in the descriptor. We'll see the code that tests that shortly.

A NOOP is especially useful for probing because it avoids moving a payload while still writing a completion record when configured to do so. That gives the attacker a simple way to exercise its own completion slot. Before looking at the experiments, let's make the required sharing conditions explicit.

Slide 9: The sharing assumptions for each channel

The two channels require different kinds of sharing. For the translation-cache channel, the attacker and victim need access to the same engine. Their work queues can be different. For the queue channel, they need to share the same shared work queue, or SWQ.

These are assumptions about the deployment. The paper gives the attacker provisioned access to DSA and explicitly requires the CAP_SYS_RAWIO capability for descriptor submission. It doesn't give the attacker authority to reconfigure the host's engines and queues. So we shouldn't describe this as an attack available to every ordinary process by default.

Within that setup, the VMs don't share memory, and the attack doesn't require a shared CPU core. The attacker observes its own submissions and completions. With those conditions clear, we can look at the experiments that establish the cache's capacity and sharing boundary.

Slide 10: One conflicting page can replace a slot

This is the capacity experiment. The first call uses base and puts its translation into the tested slot. The second call uses base plus an offset. The third call returns to base so the researchers can check whether its translation survived.

When the second address stays within the same four-kilobyte page, the final access reuses the translation. When it moves to another page, the final access misses. Across their experiments, that behavior supports a single cached entry for the field: one conflicting page is enough to replace it.

For the NOOP helper, base means the completion-record address. It isn't a victim buffer that the attacker somehow knows. The code shown here is a short experimental excerpt, so the descriptor setup and completion flags aren't included. This establishes capacity within a field. Next, we need to check whether different fields share that same entry.

Slide 11: Source and destination use separate slots

The first listing asks whether changing the source also destroys the destination's cached translation. It copies from one source to a destination, then changes the source page while keeping the destination the same. The destination still contributes a hit. That tells us the source and destination have independent cached state.

The second listing tests the overlapping descriptor position we saw earlier. It compares two buffers, then performs a copy. The second source from the compare and the destination from the copy are placed on the same page. If those two roles shared an entry, that page could provide a reuse hit in the copy.

Instead, the reported reuse comes only from the unchanged source role. The second-source translation doesn't act as a destination translation. So it is the field's meaning that matters, not just the address or its byte position in the descriptor. Now we can test which engine shares those slots.

Slide 12: Eviction follows the shared engine

The authors now move from one engine's behavior to interference between two virtual machines. They run three configurations shown in the original figure.

In E0, both sides use the same queue and engine. The attacker's primed translation is replaced, so its later probe misses. In E1, they use different queues but still reach the same engine. The probe still misses. Queue separation alone hasn't separated the translation-cache state.

E2 changes the engine as well. This time, the primed translation survives and the probe hits. That's the control that ties the observed interference to the shared engine. It supports a concrete boundary in the tested configuration; it isn't a guarantee about every accelerator or every deployment.

These characterization experiments explain what is shared. The next question is whether an attacker can recognize a hit or miss using a signal available from its own code.

Slide 13: How a probe distinguishes a hit from a miss

The attacker doesn't get a message saying which cache entry the victim touched. It measures the time taken by its own request. In the paper's measurements, requests whose translations hit cluster around five hundred cycles. Misses take more than a thousand cycles.

That separation lets the attacker choose a threshold and classify a probe as likely hit or likely miss. The authors report thresholds between six hundred and nine hundred cycles in these experiments. Those are measured values for their setup, not universal constants to copy onto any machine.

The figures also include local and cloud runs with background load. The distributions move, but remain distinguishable in those tests. Performance counters help the researchers characterize the hardware; the attack's observation is its own completion timing. A single slow request still has possible competing explanations. Repeated, calibrated measurements are what make the next attack sequence useful.

Slide 14: DevTLB attack: prime, wait, probe

Now we can put the attack together. First, I submit a completion-enabled NOOP using my own completion page. That primes the completion slot. Then I wait while the victim may run on the same engine. Finally, I submit another probe using my page and time its completion.

The code excerpt shows the timing pattern. ENQCMD submits the descriptor first. The first timestamp comes after that submission. The loop waits for the completion status to change, and the second timestamp ends the measurement. Subtracting the two gives the polling interval shown here.

There's an important source detail: this excerpt comes from the paper's compare helper. The attack uses NOOP with this timing approach; the slide isn't claiming that the displayed excerpt is its complete implementation. A fast probe suggests retention, while a slow probe suggests replacement. We now have an activity signal. The other channel uses queue admission feedback instead of a duration.

Slide 15: Why use a long ordinary copy?

To observe queue contention, the attacker needs to keep work outstanding long enough to create an observation window. A batch might seem like an easy way to do that, but its scheduling priority matters.

In this experiment, the authors submit a batch descriptor and an ordinary work descriptor to the same queue. The fence separates the submissions, and the code times completion of the ordinary work. They repeat the experiment after swapping the two submission positions.

The ordinary work's completion latency stays nearly unchanged. Together with the other tested cases, that supports a preference for ordinary work over batch-buffer work. A batch therefore isn't a reliable way to hold up the ordinary requests the attacker wants to observe.

This listing is a scheduling experiment, not the congestion attack itself. Its result explains the next design choice: use one long ordinary copy to keep the queue busy during the probe window.

Slide 16: SWQ attack: fill, wait, probe

Here is the second attack. The first submission is the long copy. The loop then submits queue-size minus two additional descriptors. The aim is to keep the queue near full while leaving one admission opportunity during the observation window.

The attacker waits, giving the victim a chance to submit. Then the probe helper tries to submit one more descriptor. This time the observation is the submission result. A zero ZF means the request was accepted. A set ZF means retry, corresponding to a full queue in this experiment.

Under the prepared conditions, a retry is evidence that another submission consumed the remaining capacity. It isn't proof of a particular victim operation by itself. Work can drain, and other activity can interfere, so the window must stay controlled and shorter than the anchor copy's execution.

The paper observed contention with copies of at least thirty-two mebibytes. The signal needs no duration measurement, but it actively congests a resource the victim also uses.

Slide 17: From a probe result to an inference

At this point, neither channel has produced a website name or a character. The raw data is much simpler: a duration from my request, or an accepted-or-retry result from my submission.

The next step is to turn those measurements into a time series. For the timing channel, a calibrated threshold marks likely misses. The attacker can count those events within fixed windows and record how the counts change over time. Queue feedback can likewise reveal when contention appears.

The meaning comes from the workload and the analysis. In a classification experiment, the attacker first collects examples with known labels and trains a model to recognize their patterns. Keystroke timing instead looks for candidate events and their timestamps.

So the hardware never tells the attacker, 'This is a browser.' The experiment supplies that context, and the analysis supplies the inferred label. We'll first see a cooperative communication test, then the application cases.

Slide 18: Covert communication through shared state

A covert channel is a controlled communication experiment. Here, the sender cooperates with the receiver: it creates activity to represent one bit value and stays idle to represent the other. The receiver uses its probes to decode those changes.

The reported DevTLB channel reaches seventeen point one nine kilobits per second, with four point six three percent bit error. The SWQ channel reaches four point zero two kilobits per second, with a higher error rate.

The plot shows why increasing the raw signaling rate doesn't always help. Faster signaling can create more errors and reduce useful capacity. This establishes a communication channel under coordinated conditions. Next, the sender stops cooperating: ordinary website traffic becomes the source of activity.

Slide 19: Why website traffic reaches DSA

This figure connects the hardware channel to the website experiment. The victim application runs in one VM, and the spy runs in another. The network path uses VPP and memif, with DSA accelerating packet movement.

Loading a website creates a sequence of network transfers. Those transfers generate accelerator work, and that work can replace translations in the engine the spy shares. The spy continues to measure its own probes, so the resulting trace reflects changes in accelerator activity over time.

There is no packet payload, URL, or browser-process name in that observation. The attacker knows the experimental task is browsing and has collected examples from known websites. That context is what lets a later classifier associate a trace with a site.

Now let's look at how the paper builds those features and what its accuracy numbers actually describe.

Slide 20: How a trace becomes a website label

The spy samples every ten microseconds and groups four hundred samples into a four-millisecond slot. Each slot records a miss count. About two hundred and fifty slots form a trace, so the feature is the pattern of activity across roughly a second.

The dataset contains one hundred websites with two hundred traces per site, giving twenty thousand traces. The authors use an eighty-twenty training and test split and an attention-based bidirectional LSTM to map those traces to website labels.

The headline result is eighty-five point seven three percent on all one hundred sites. The confusion matrix shown here covers only fifteen selected sites, where the reported accuracy is ninety-six point five percent. Those are different evaluation scopes.

This is a closed-world task: training and testing use the same known set of website labels. It doesn't establish recognition of arbitrary unseen websites. The next application uses the timing of individual events rather than a whole-page loading pattern.

Slide 21: Keystrokes reveal events and timing

For the SSH experiment, the victim uses DSA Transparent Offload, or DTO. It redirects eligible memory operations to DSA, so handling interactive SSH traffic can generate accelerator activity. The attacker looks for that activity and estimates when candidate keystroke events occurred.

The plots are filtered traces. The letters shown on them are known input labels used to evaluate the experiment. They are not characters decoded from the cache. The demonstrated result is event detection and timing, not recovery of the command text.

The paper reports an F1 score of ninety-two percent for DevTLB and ninety-eight point four percent for SWQ. The standard deviations of timestamp differences are five point two nine milliseconds and one point two one milliseconds, respectively.

There is a reporting caveat: the printed detection, false-positive, and false-negative counts don't fully reconcile with those metrics. We therefore keep the headline scores labeled as reported. The final application asks which model configuration produced an activity pattern.

Slide 22: Fingerprinting model configurations

Model inference also moves substantial amounts of data. With the evaluated offload setup, those memory operations create DSA activity. Different model configurations can therefore produce different patterns in the probe trace.

The paper tests eight configurations across four groups: TinyStories, LLaMA 2, Gemma 3, and Qwen3. It collects fifty traces per configuration, uses identical prompts, and splits the data eighty-twenty for training and validation. The aggregation window is eight milliseconds.

Within that task, the reported validation accuracy is ninety-eight point six percent. The inferred label is one of the known model configurations. The attacker doesn't recover model weights or read the prompt.

The fixed prompts and evaluated runtimes matter when interpreting that number. The result shows that the configurations are distinguishable within this dataset; it doesn't establish identification across arbitrary prompts, hosts, or unseen models. We can now ask how these results change under the tested background load.

Slide 23: Robustness within the tested setup

This table brings the results together across local and cloud settings, with and without injected background load. Most reported metrics change modestly across those conditions. The DevTLB keystroke F1 varies more, ranging from ninety point five to ninety-three point four percent.

The useful conclusion is that the channels remained effective under these tested stressors. The experiments include PCIe and memory pressure, so the results aren't limited to an entirely idle system.

But that isn't the same as testing every source of interference on a busy shared host. Many independent tenants, changing workloads, or different provisioning could create additional challenges. With that scope in mind, let's look at what the paper proposes to reduce the leakage.

Slide 24: Mitigations and what remains unmeasured

The practical boundary follows directly from the experiments. Separate queues can still share an engine's translation state, so protecting against this DevTLB channel requires attention to engine assignment. For the SWQ channel, avoid sharing the same queue across trust boundaries. Disabling offload on a sensitive path is another possible tradeoff.

The paper also proposes hardware changes, including partitioning translation-cache replacement state by address-space context and restricting queue-state feedback. Those are proposals, not evaluated deployed fixes, and they come with performance or interface costs.

For a software mitigation based on inserting random translations, the authors measure throughput loss. The maximum reported losses are fifteen point seven percent for native DSA and seventeen point nine percent with DTO, at 256-byte transfers. What they don't quantify is the leakage that remains after mitigation. So we have a cost measurement, but not a complete security-performance result. Let's close with what the experiments do establish.

Slide 25: What the experiments establish

The paper establishes two observable channels in shared DSA resources: translation replacement affects completion timing, and shared-queue admission reveals contention. Controlled experiments connect those observations to the engine and queue boundaries.

The application results then show how repeated observations can support communication, website classification, keystroke timing, and model-configuration classification under the evaluated conditions. The hardware supplies an activity signal; the workload and analysis give it meaning.

The broader lesson is that protecting addresses doesn't automatically hide activity in shared accelerator state. Both need to be considered when defining isolation. Thank you. We're happy to take questions.